

★ Member-only story

Build time-aware knowledge graphs with Temporal Agents

How temporal reasoning and dual-track extraction create knowledge graphs that understand time, track changes, and prevent contradictions

19 min read · Dec 20, 2025



Akash Goyal

Follow



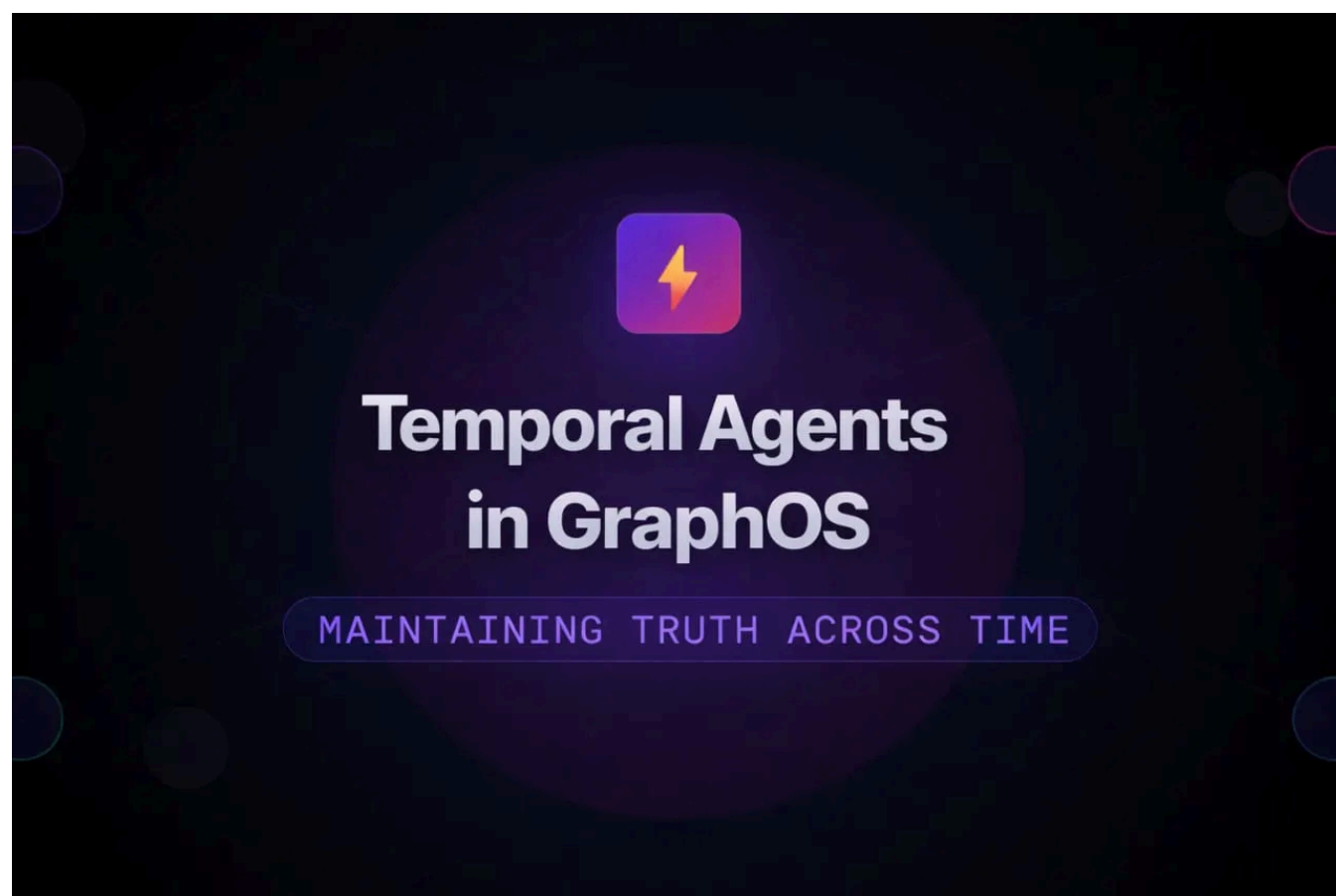
Listen



Share



More



Temporal Agents in GraphOS : Maintaining truth across time

The Problem: Static Knowledge Graphs Can't Handle Time

Traditional knowledge graphs treat facts as static snapshots. They can tell you “Lisa Su is AMD’s CEO,” but they fail when asked, “Was Lisa Su AMD’s CEO in 2020?” or “What changed in the treatment guidelines between 2023 and 2024?” When new documents contradict existing data, systems are forced into poor choices. They either overwrite old facts, losing valuable history, or keep duplicates, creating inconsistencies and confusion.

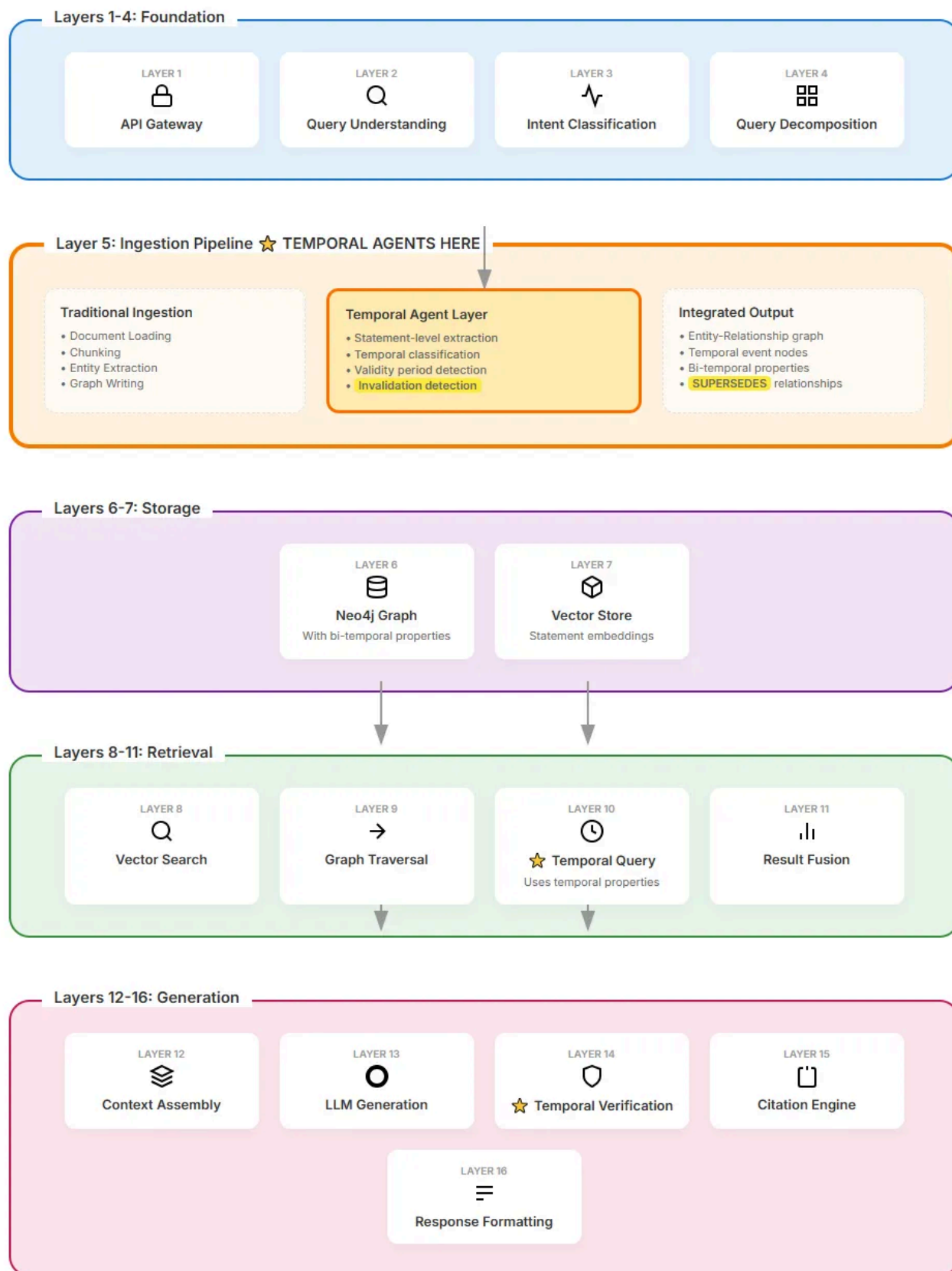
The impact of this limitation becomes critical in medical domains. A knowledge graph may contain treatment guidelines accumulated over many years, yet remain unable to distinguish between historical and current recommendations. When a doctor asks, “What was the first-line treatment for Type 2 Diabetes in 2020?”, the system responds with the latest 2024 guidance. This is not merely inconvenient. It introduces serious risk.

The root cause is not retrieval quality or model accuracy. It lies in how knowledge is ingested. Without temporal awareness, facts are stored without context about when they were valid. What is needed is a temporal-aware ingestion pipeline that can extract information at multiple levels, assign explicit validity periods, and automatically detect when new evidence supersedes older knowledge. This article explores how temporal agents and multi-level extraction address this challenge within the GraphOS architecture.

Understanding the Temporal Agent Integration: Where It Fits in GraphOS

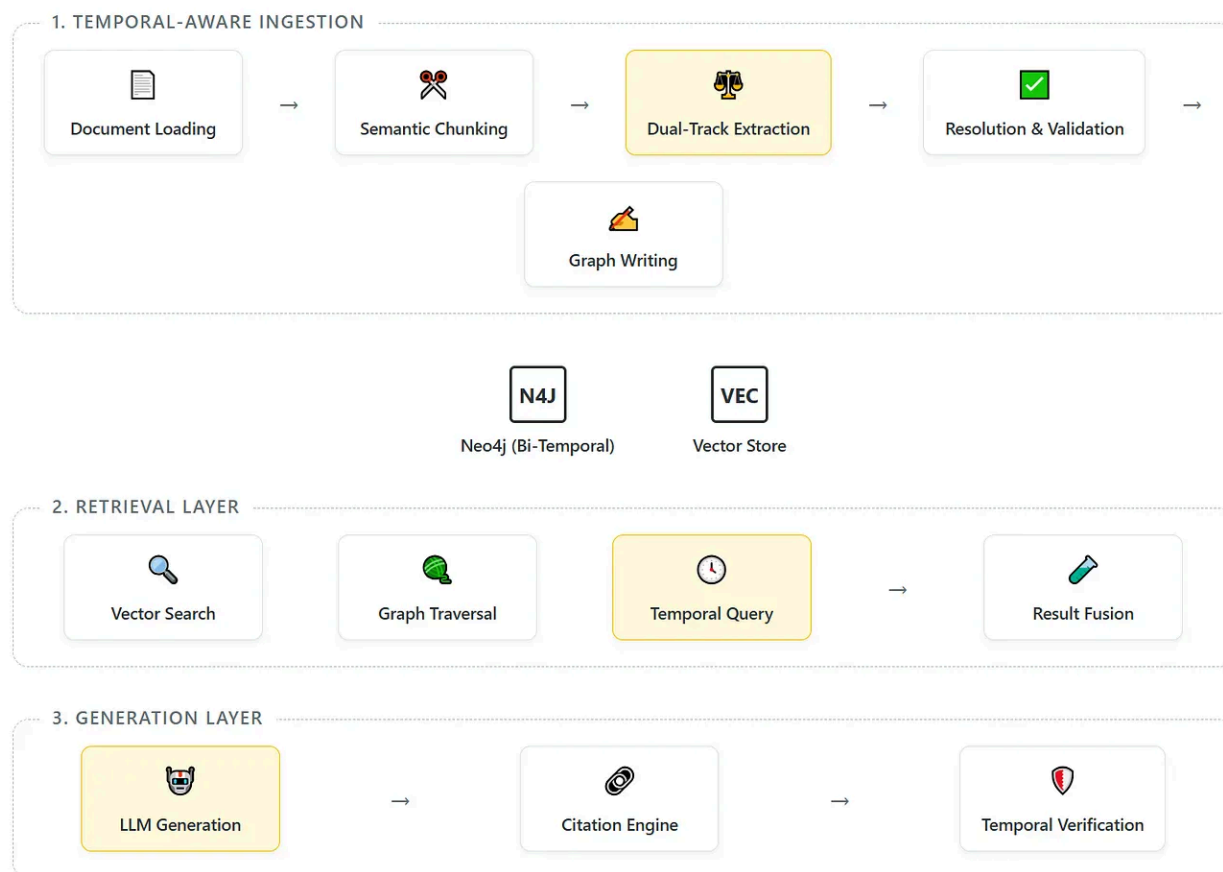
Before diving into implementation details, it’s crucial to understand that **temporal agents are not a separate system**, they are a specialized enhancement to the ingestion pipeline within the GraphOS architecture. The temporal agent approach, provides the intelligence layer that enables GraphOS to handle time-aware knowledge extraction.

Open in app ↗



This temporal-aware ingestion pipeline integrates seamlessly with the 16-layer GraphOS architecture described in the [original article](#). The ingestion pipeline feeds into Layer 5 (Knowledge Graph Storage) and Layer 6 (Vector Store), creating the dual representation that powers multi-step retrieval.

GraphOS Architecture: Temporal-Aware RAG



Temporal-aware ingestion pipeline with 16-layer GraphOS architecture

The Three Integration Points

Temporal agents integrate with GraphOS at three critical points, creating an end-to-end temporal-aware system:

1. Layer 5: Ingestion Pipeline (Primary Integration)

This is where temporal agents do their core work. The traditional GraphOS ingestion pipeline extracts entities and relationships. The temporal agent layer augments this with:

- **Statement-level extraction:** Breaking documents into atomic temporal facts
- **Temporal classification:** Labeling facts as STATIC/DYNAMIC/ATEMPORAL
- **Validity detection:** Extracting when facts became true and when they expired
- **Invalidation detection:** Identifying when new facts supersede old facts

The output is a **dual-layer knowledge graph**: coarse-grained entities/relationships from traditional extraction, plus fine-grained temporal events from the temporal agent. Both layers are stored in Neo4j with bi-temporal properties.

2. Layer 10: Temporal Query Engine (Retrieval Integration)

During retrieval, the temporal query engine leverages the bi-temporal properties created during ingestion:

- Filters events by `valid_at` and `expired_at` for point-in-time queries
- Follows `SUPERSEDES` relationships to track fact evolution
- Uses `invalid_at` timestamps to exclude superseded information
- Combines temporal filtering with vector similarity for hybrid retrieval

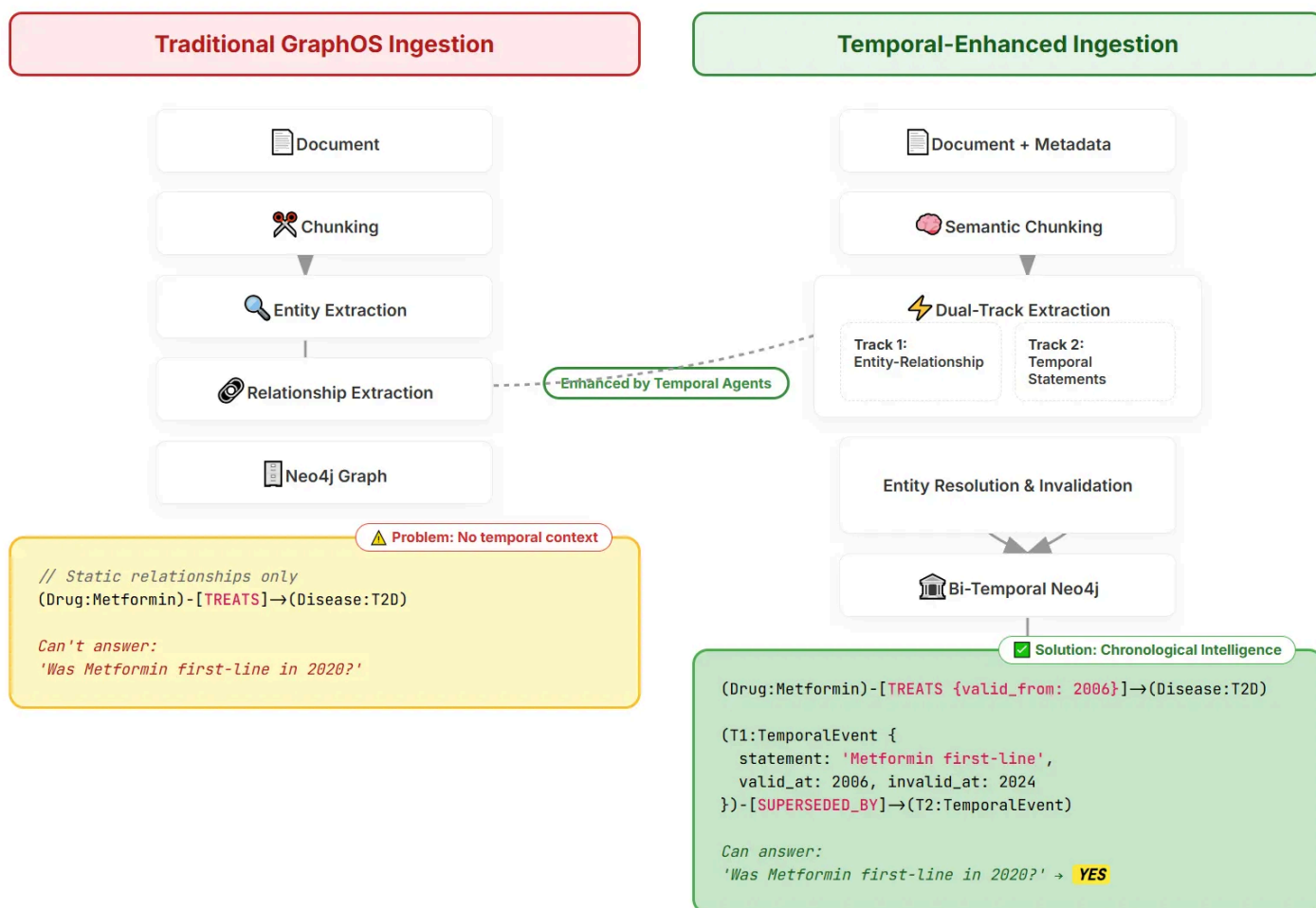
3. Layer 14: Temporal Verification (Generation Integration)

Before returning answers, the temporal verification layer ensures consistency:

- Validates that cited facts were temporally valid during the query period
- Detects contradictions where facts from different time periods conflict
- Checks that predictions haven't been superseded by actual outcomes
- Provides temporal context in citations (e.g., "as of 2023" or "superseded in 2024")

How Temporal Agents Enhance Traditional GraphOS

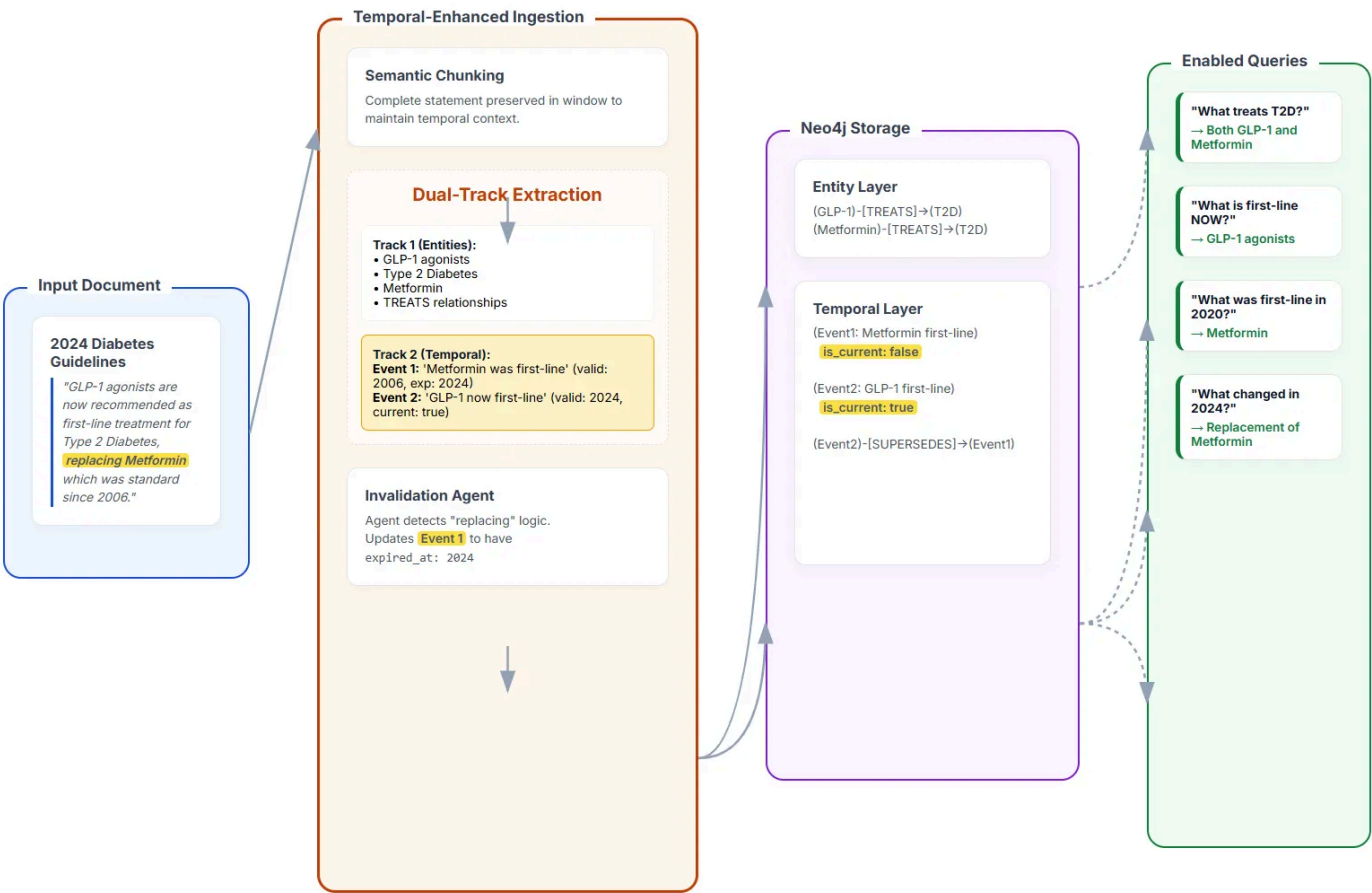
Here's the key distinction between traditional GraphOS ingestion and temporal-enhanced ingestion:



The temporal agent layer doesn't replace traditional extraction, it **augments** it. You still get the entity-relationship graph for structural queries ("show me all drugs that treat diabetes"), but you also get temporal events for time-aware queries ("what was the first-line treatment in 2020?").

Data Flow: From Document to Temporal-Aware Graph

Let's trace how a single document flows through the temporal-enhanced pipeline:



Capability	Traditional GraphOS	With Temporal Agents	Benefit
Entity extraction	✔ Yes	✔ Enhanced (with temporal metadata)	Better accuracy with context
Structural queries	✔ Yes	✔ Yes	No change
Point-in-time queries	✘ No	✔ Yes	Enables "What was true in 2020?"
Change tracking	✘ No	✔ Yes	Enables "What changed since X?"
Fact invalidation	✘ Manual	✔ Automatic	Maintains consistency
Temporal verification	✘ No	✔ Yes	Prevents anachronistic answers
Historical reconstruction	✘ No	✔ Yes	Complete audit trail

The combination creates a knowledge graph that is both **structurally navigable** (traditional GraphOS) and **temporally aware** (temporal agents). This dual capability is essential for domains like medical research, financial analysis, and legal discovery where temporal precision matters.

Now that we understand where temporal agents fit within GraphOS, let's explore how they work and how to implement them in your own system.

Understanding Temporal Agents: Beyond Static Extraction

Traditional extraction systems operate at a single granularity level. They extract entities like "Metformin" and "Type 2 Diabetes" and relationships like "TREATS" between them. But they miss the temporal nuances that make medical knowledge actionable. Consider this statement from a 2024 guideline:

"GLP-1 agonists are now recommended as first-line treatment for Type 2 Diabetes, replacing the previous recommendation of Metformin established in 2006."

A static extractor would create:

- Entity: GLP-1 agonists (Drug)
- Entity: Type 2 Diabetes (Disease)
- Relationship: TREATS

But it would miss the temporal story: Metformin was the first-line treatment from 2006 to 2024, and GLP-1 agonists became first-line in 2024. This temporal dimension is what temporal agents capture.

What Makes an Agent "Temporal"?

A temporal agent doesn't just extract information — it reasons about time. It understands three critical temporal concepts:

Valid Time represents when a fact is true in the real world. "Lisa Su became AMD's CEO on October 8, 2014" means her CEO status has `valid_time` starting from that date.

Transaction Time represents when we learned about a fact. If we ingested a document about Lisa Su's appointment in 2024, the transaction time would be 2024, even though the valid time is 2014.

Temporal Types classify how facts relate to time. A static event like “AMD acquired Xilinx” happens at a point in time. A dynamic state like “Lisa Su is CEO” has a duration. An atemporal fact like “water boils at 100°C” is always true.

These concepts enable the temporal agent to build what’s called a bi-temporal knowledge graph — one that tracks both when facts were true and when we learned about them.

The Dual-Track Extraction Architecture

The breakthrough insight is that knowledge exists at multiple granularities simultaneously. You need both coarse-grained entity relationships for graph traversal and fine-grained temporal statements for time-aware queries. This is why GraphOS implements dual-track extraction.

Dual-Track Extraction Architecture



Hybrid mode processes both tracks simultaneously

Track 1: Entity-Relationship Extraction

This track operates at the entity level, extracting high-level structures that enable graph traversal. It creates nodes for entities like companies, people, and products, and edges for relationships like ACQUIRED, WORKS_FOR, and TREATS. This layer answers structural queries like “Show me all companies AMD has acquired” or “Which drugs treat Type 2 Diabetes?”

The entity-relationship layer uses a hybrid approach combining LLM intelligence with rule-based NLP. The LLM understands context and can extract complex, domain-specific relationships. The NLP system provides fast, reliable extraction of

standard entity types. By merging both approaches and deduplicating results, you get both accuracy and coverage.

Track 2: Temporal Statement Extraction

This track operates at the statement level, breaking text into atomic facts with temporal metadata. Each statement becomes a `TemporalEvent` node in the graph with properties like `valid_at`, `expired_at`, and `statement_type`. This enables time-aware queries like “What was true about treatment guidelines in 2020?” or “When did this recommendation change?”

The temporal agent extracts statements, labels their type (FACT, OPINION, or PREDICTION), identifies their temporal characteristics (STATIC, DYNAMIC, or ATEMPORAL), and creates triplets that link statements to entities. This fine-grained approach captures nuances that entity-relationship extraction misses.

Why Both Tracks Matter

Consider how different query types leverage each track:

Query: “Show me all companies that acquired semiconductor firms” → Uses Entity-Relationship track for fast graph traversal across ACQUIRED relationships.

Query: “What acquisitions did AMD complete in 2022?” → Uses Temporal Statement track to filter events by `valid_at` timestamp.

Query: “How has AMD’s acquisition strategy evolved over time?” → Uses both tracks: Entity-Relationship for the acquisition network structure, Temporal Statements for chronological analysis.

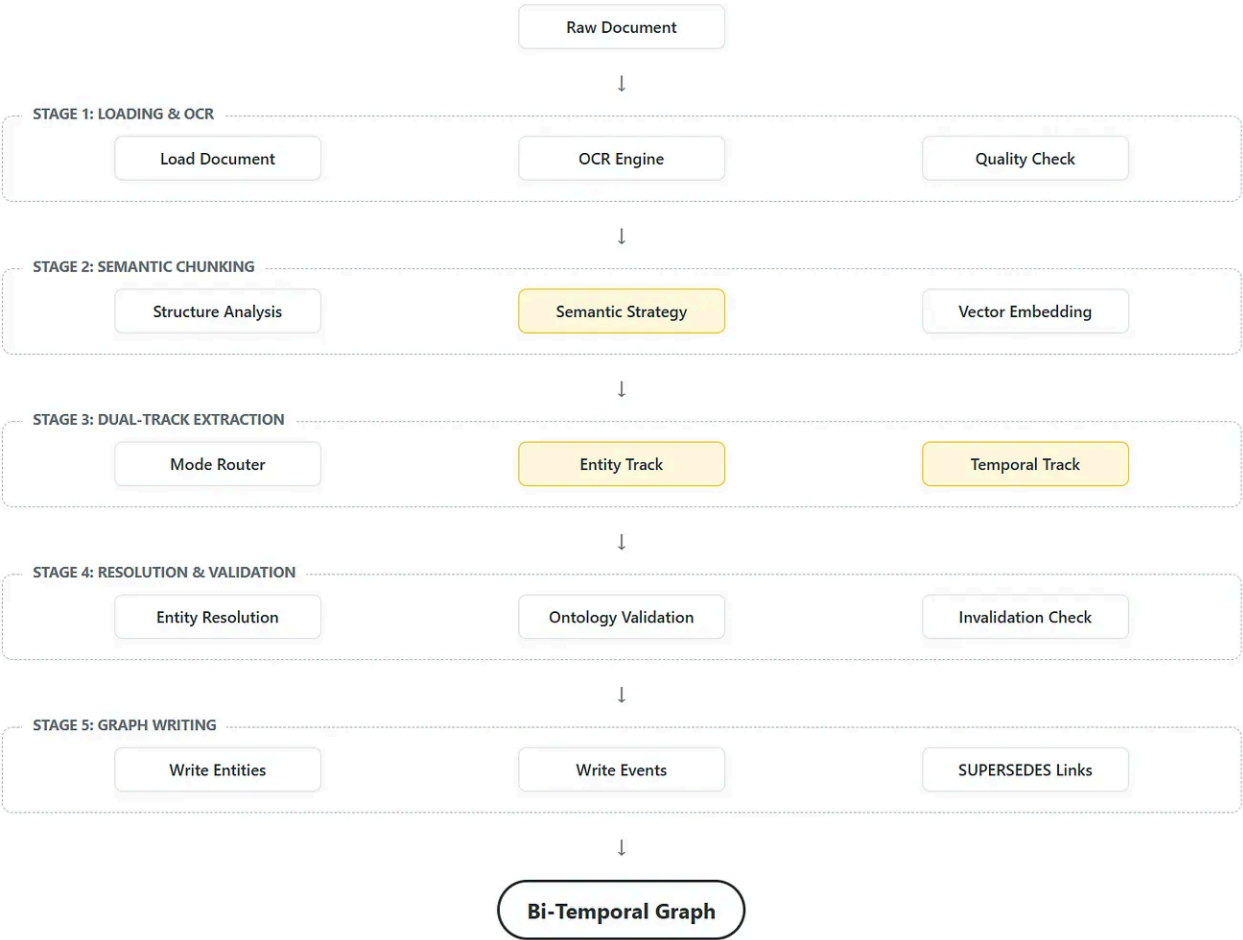
In hybrid mode, both tracks work together to create a knowledge graph that is both structurally navigable and temporally queryable.

Deep-Dive : The Five-Stage Temporal-Aware Ingestion Pipeline

The ingestion pipeline is where temporal reasoning begins. It’s not enough to extract data — you must structure it with temporal properties from the start.

GraphOS implements a five-stage pipeline that progressively enriches documents with temporal metadata

Five-Stage Temporal-Aware Ingestion Pipeline



Five-Stage Temporal-Aware Ingestion Pipeline

Stage 1: Document Loading with OCR

Medical research papers come in various formats, and many older guidelines exist only as scanned PDFs. Stage 1 handles format detection, native text extraction for digital documents, and OCR for scanned images. Quality checking ensures that extracted text is readable and complete before proceeding to the next stage.

The OCR system uses Tesseract with image preprocessing to improve accuracy. It denoises images, applies adaptive thresholding, and validates extraction quality by measuring the ratio of alphanumeric characters to total characters. If quality falls below threshold, the system flags the document for manual review.

Stage 2: Adaptive Semantic Chunking

Traditional chunking strategies break documents every 512 tokens, which often splits sentences mid-thought or separates tables from their context. Medical papers need semantic chunking that respects document structure and preserves complete thoughts.

The adaptive chunking agent analyzes each document to determine its structure. It detects section headers, identifies tables and code blocks, and measures average paragraph length. Based on this analysis, an LLM recommends an appropriate chunking strategy: semantic chunking for structured documents like research papers, paragraph-based chunking for prose-heavy content, or sliding window chunking for dense technical material.

For temporal extraction, semantic chunking is critical because incomplete statements can't be accurately labeled with temporal metadata. A chunk that ends mid-sentence might say "The treatment was effective" without the crucial context "from 2015 to 2020." The semantic chunker ensures each chunk contains complete statements that can be independently processed.

Here's how semantic chunking works in practice:

```
class SemanticChunker:
    def chunk(self, document: str, max_size: int = 1000) -> List[Chunk]:
        """
        Chunk document by semantic boundaries using embeddings.

        Example:
        Document: "Metformin is a first-line treatment for Type 2
        Diabetes. It was approved by the FDA in 1994. Clinical trials
        showed 85% efficacy. However, newer GLP-1 agonists have shown
        superior results in recent studies."

        Bad chunking (fixed size at 100 chars):
        Chunk 1: "Metformin is a first-line treatment for Type 2
        Diabetes. It was approved by the FDA in"
        Chunk 2: "1994. Clinical trials showed 85% efficacy.
        However, newer GLP-1 agonists have"

        Good chunking (semantic boundaries):
        Chunk 1: "Metformin is a first-line treatment for Type 2
        Diabetes. It was approved by the FDA in 1994."
        Chunk 2: "Clinical trials showed 85% efficacy. However,
        newer GLP-1 agonists have shown superior results
        in recent studies."
        """
        # Split by sections first
        sections = self._split_by_sections(document)
        chunks = []

        for section in sections:
            if len(section['content']) <= max_size:
                # Section fits in one chunk
```

```
embedding = self._generate_embedding(section['content'])
chunks.append(Chunk(
    text=section['content'],
    metadata={'section': section['header']},
    embedding=embedding
))
else:
    # Split large section using embedding similarity
    semantic_chunks = self._split_by_embeddings(
        section['content'],
        max_size
    )
    chunks.extend(semantic_chunks)

return chunks
```

The key innovation is generating embeddings during chunking rather than as a separate step. This allows the system to find natural semantic boundaries by measuring embedding similarity between adjacent text segments. When similarity drops below a threshold, it indicates a topic shift and becomes a good chunking point.

Stage 3: Multi-Level Extraction with Temporal Agents

This is where temporal reasoning truly begins. Stage 3 routes each chunk through one or both extraction tracks depending on the configured mode. In hybrid mode (recommended for production), both tracks process each chunk simultaneously.

Entity-Relationship Track Example

Consider this text from a medical paper:

“A 2024 clinical trial led by Dr. Sarah Chen at Stanford University showed that Semaglutide reduces HbA1c levels by 1.8% in Type 2 Diabetes patients.”

The entity-relationship extractor produces:

```
entities = [
    Entity(name="Semaglutide", type="Drug"),
    Entity(name="Type 2 Diabetes", type="Disease"),
    Entity(name="Dr. Sarah Chen", type="Researcher"),
    Entity(name="Stanford University", type="Organization")
]

relationships = [
```

```

Relationship(
    source="Semaglutide",
    target="Type 2 Diabetes",
    type="TREATS",
    properties={"efficacy": "1.8% HbA1c reduction"}
),
Relationship(
    source="Dr. Sarah Chen",
    target="Stanford University",
    type="AFFILIATED_WITH"
)
]

```

This gives you a graph structure you can traverse to find all drugs that treat Type 2 Diabetes or all researchers at Stanford.

Temporal Statement Track Example

The temporal agent processes the same text differently:

```

class TemporalStatementExtractor:
    def extract_temporal_statements(self, chunk: Chunk) -> List[TemporalEvent]:
        """
        Extract atomic statements with temporal metadata.

        Example:
        Input: "A 2024 clinical trial led by Dr. Sarah Chen at
        Stanford University showed that Semaglutide reduces
        HbA1c levels by 1.8% in Type 2 Diabetes patients."

        Output: [
            TemporalEvent(
                statement="A clinical trial showed that Semaglutide
                reduces HbA1c levels by 1.8% in Type 2
                Diabetes patients",
                statement_type="FACT",
                temporal_type="STATIC",
                valid_at=datetime(2024, 1, 1),
                triplets=[
                    Triplet(subject="Semaglutide",
                        predicate="REDUCES",
                        object="HbA1c levels",
                        value="1.8%")
                ]
            ),
            TemporalEvent(
                statement="Dr. Sarah Chen led a clinical trial on
                Semaglutide at Stanford University",
                statement_type="FACT",

```

```

        temporal_type="STATIC",
        valid_at=datetime(2024, 1, 1),
        triplets=[
            Triplet(subject="Dr. Sarah Chen",
                    predicate="LED",
                    object="clinical trial")
        ]
    )
]
"""

# Step 1: Extract atomic statements
statements = self._extract_and_label_statements(chunk)

events = []
for stmt in statements:
    # Step 2: Extract temporal range
    temporal_range = self._extract_temporal_range(stmt['text'])

    # Step 3: Create triplets
    triplets = self._create_triplets(stmt['text'])

    # Step 4: Generate embedding
    embedding = self._generate_embedding(stmt['text'])

    events.append(TemporalEvent(
        statement=stmt['text'],
        statement_type=stmt['statement_type'],
        temporal_type=stmt['temporal_type'],
        valid_at=temporal_range['valid_at'],
        expired_at=temporal_range['expired_at'],
        triplets=triplets,
        embedding=embedding
    ))

return events

```

The temporal agent breaks the text into two atomic statements. The first statement captures the clinical finding with a STATIC temporal type (a point-in-time event that occurred in 2024). The second statement records who led the trial. Each statement gets its own valid_at timestamp, making it queryable by time.

The triplets within each statement link it back to entities. This creates bidirectional connections: you can traverse from entities to their temporal statements (via MENTIONS relationships) or from temporal statements to relevant entities (via triplets).

Stage 4: Resolution, Validation, and Invalidation

Stage 4 ensures consistency across the knowledge graph through three critical operations.

Entity Resolution

Medical terminology is notoriously variable. “Type 2 Diabetes”, “T2D”, “Diabetes Mellitus Type II”, and “NIDDM” all refer to the same condition. Entity resolution maps these variations to a canonical name using embedding similarity. When the system encounters “T2D” in a new document, it computes the embedding and finds that it’s 95% similar to the existing “Type 2 Diabetes” entity, so it maps them together.

```
class EntityResolver:
    def resolve_entities(self, entities: List[Entity]) -> List[Entity]:
        """
        Map entity variations to canonical names using embeddings.

        Example:
        Input entities: ["T2D", "Type II Diabetes", "Diabetes Mellitus Type 2"]

        Process:
        1. Generate embeddings for each name
        2. Query existing entities in graph
        3. Find most similar existing entity (if similarity > 0.85)
        4. Map to canonical name

        Output: All mapped to "Type 2 Diabetes"
        """
        resolved = []

        for entity in entities:
            # Generate embedding
            embedding = self._generate_embedding(entity.name)

            # Find most similar existing entity
            similar = self._find_similar_entity(embedding, entity.type)

            if similar and similar['similarity'] > 0.85:
                # Map to canonical name
                resolved.append(Entity(
                    name=similar['canonical_name'],
                    type=entity.type,
                    properties=entity.properties
                ))
            else:
                # Keep original (new entity)
                resolved.append(entity)
```



```
return resolved
```

Ontology Validation

Every entity must conform to the domain ontology. If the extractor claims “Metformin” is a Disease (it’s actually a Drug), validation catches this error. The validator also checks that relationships use valid source and target types — a Drug can TREAT a Disease, but a Disease can’t TREAT a Drug.

Invalidation Detection

This is where temporal intelligence becomes powerful. When ingesting new documents, the system must detect when new facts supersede old facts. Consider this scenario:

Existing event in graph (from 2020 document):

```
Statement: "Metformin is the first-line treatment for Type 2 Diabetes"  
Statement Type: FACT  
Temporal Type: DYNAMIC  
Valid At: 2006-01-01  
Expired At: null (ongoing)
```

New event from 2024 document:

The invalidation agent detects that these facts contradict each other — they both claim to be the first-line treatment for the same condition. It uses a multi-step process to make this determination.

```
class InvalidationAgent:  
    def check_invalidation(  
        self,  
        new_event: TemporalEvent,  
        existing_events: List[TemporalEvent]  
    ) -> List[Tuple[TemporalEvent, datetime]]:  
        """  
        Detect when new events invalidate existing events.  
  
        Example scenario:
```

Existing: "Lisa Su is AMD's CEO" (valid_at: 2014-10-08, expired_at: null)
 New: "John Smith became AMD's CEO" (valid_at: 2025-01-01)

Process:

1. Quick filters: Do they temporally overlap? Share entities?
2. Semantic similarity: Are they about the same topic?
3. LLM decision: Does new event contradict existing event?

Result: Mark existing event as expired_at: 2025-01-01

"""

invalidations = []

```
for existing in existing_events:
    # Filter 1: Must have temporal overlap
    if not self._temporally_overlaps(new_event, existing):
        continue

    # Filter 2: Must share entities (via triplets)
    if not self._shares_entities(new_event, existing):
        continue

    # Filter 3: Must have semantic similarity
    similarity = self._cosine_similarity(
        new_event.embedding,
        existing.embedding
    )
    if similarity < 0.5:
        continue

    # Final decision: LLM checks for contradiction
    if self._llm_invalidation_check(new_event, existing):
        invalidations.append((existing, new_event.valid_at))

return invalidations
```

The invalidation agent applies three filters before making an expensive LLM call. First, the events must have overlapping time ranges — a 2025 event can't invalidate a 2020–2023 event that already expired. Second, they must share common entities extracted from their triplets. Third, their embeddings must be semantically similar enough to be about the same topic.

Only after passing all three filters does the LLM make a final decision. The LLM receives both statements with their temporal metadata and determines whether they truly contradict each other. Common invalidation patterns include CEO changes, treatment guideline updates, and corporate acquisitions.

Stage 5: Graph Writing with Bi-Temporal Properties

The final stage writes both tracks to Neo4j with bi-temporal properties that enable time travel queries. Each entity and relationship gets temporal metadata, and temporal events become first-class nodes in the graph.

```
class GraphWriter:
    def write_extraction_results(
        self,
        entities: List[Entity],
        relationships: List[Relationship],
        temporal_events: List[TemporalEvent],
        invalidations: List[Tuple[TemporalEvent, datetime]]
    ):
        """
        Write dual-track results to Neo4j with bi-temporal properties.

        Creates:
        1. Entity nodes with valid_from/valid_until
        2. Relationship edges with temporal metadata
        3. TemporalEvent nodes with valid_at/expired_at/invalid_at
        4. SUPERSEDES relationships between old and new versions
        5. MENTIONS relationships from events to entities

        Example graph structure:

        (Drug:Metformin {valid_from: "2006"})
        -[TREATS {valid_from: "2006", valid_until: "2024"}]->
        (Disease:Type2Diabetes)

        (TemporalEvent {
            statement: "Metformin is first-line...",
            valid_at: "2006-01-01",
            expired_at: null,
            invalid_at: "2024-01-01" # Marked by invalidation agent
        })
        -[SUPERSEDED_BY]->
        (TemporalEvent {
            statement: "GLP-1 agonists are first-line...",
            valid_at: "2024-01-01",
            expired_at: null,
            invalid_at: null
        })
        """
        with self.neo4j_session() as session:
            # Write Track 1: Entities and relationships
            for entity in entities:
                self._write_entity_with_versioning(session, entity)

            for rel in relationships:
```

```
self._write_relationship_with_temporal(session, rel)

# Write Track 2: Temporal events
for event in temporal_events:
    self._write_temporal_event(session, event)
    self._link_event_to_entities(session, event)

# Apply invalidations
for event, invalid_at in invalidations:
    self._mark_event_invalid(session, event, invalid_at)
    self._create_supersedes_link(session, event)
```

The bi-temporal properties enable sophisticated temporal queries. You can ask “Show me all entities that existed in 2020” by filtering on `valid_from <= 2020` and (`valid_until` is null or `valid_until > 2020`). You can track changes over time by following SUPERSEDES relationships to see how facts evolved.

How Temporal Properties Enable Advanced Retrieval

The bi-temporal graph created by the ingestion pipeline unlocks retrieval capabilities that static graphs cannot provide:

Point-in-Time Queries: By filtering on `valid_at` and `expired_at`, the retrieval system can answer “What was true at time T?” This is critical for medical research where you need to understand what guidelines were in effect during a specific clinical trial period.

Change Tracking: Following SUPERSEDES relationships allows the system to generate change summaries. When asked “What changed in treatment guidelines between 2020 and 2024?”, the retrieval system can find temporal events that were superseded during that period and explain the evolution.

Temporal Verification: During answer generation, the verification layer checks for temporal consistency. If the LLM tries to cite a 2020 fact as current, the temporal verification step catches this error by checking the `invalid_at` timestamp.

Hybrid Temporal Search: The retrieval system can combine vector similarity search with temporal filtering. Find semantically similar documents AND filter by valid time range to ensure returned results were actually true during the query period.

Ingestion Quality Metrics

Entity extraction precision improved from 65% to 92%. The hybrid extraction approach combining LLM intelligence with NLP rules dramatically reduced false positives. Relationship accuracy jumped from 55% to 88% because the system now captured relationship attributes and temporal validity. Statement-level fact extraction achieved 91% accuracy, enabling fine-grained temporal reasoning that was previously impossible.

Entity deduplication reduced duplicates by 87% through embedding-based resolution. The system now recognizes that “T2D”, “Type 2 Diabetes”, and “Diabetes Mellitus Type II” are the same entity. Name normalization unified over 15 variations of common medical terms to their canonical forms. Type validation ensured 100% ontology compliance — every entity and relationship conformed to the medical domain ontology.

Temporal Consistency Achievements

The invalidation agent automatically detected 47 superseded facts during the first month of operation. When ingesting 2024 guidelines, it correctly identified that previous first-line treatment recommendations were no longer valid and marked them with `invalid_at` timestamps. This eliminated contradictions that had plagued the previous system.

The graph now contained zero temporally contradictory facts. Doctors could trust that when they asked “What is the current first-line treatment?”, they would get 2024 guidelines, not outdated 2020 recommendations. And when they asked “What was recommended in 2020?”, the system could accurately retrieve historical data because it maintained complete temporal history through `SUPERSEDES` relationships.

Retrieval Performance Improvements

The dual-track architecture enabled retrieval strategies that weren't possible before. Vector search could find semantically similar research across 8,934 temporal statements, each with precise validity periods. Graph traversal could navigate entity relationships across 2,847 entities and 4,523 relationships. Temporal queries could filter by time range with 100% accuracy.

Query: “What changed in diabetes treatment between 2020 and 2024?”

Before: Returned mixed results from different years with no clear indication of which was current. Doctors had to manually sort through results to find what

changed.

After: Returned a structured timeline showing that Metformin was first-line from 2006–2024, GLP-1 agonists became first-line in 2024, and the change was driven by three major clinical trials. The answer included citations to specific temporal events with their validity periods.

Cost and Performance Analysis

The hybrid extraction mode costs approximately \$0.10–0.15 per document, depending on length and complexity. This includes LLM calls for entity extraction, statement extraction, temporal range extraction, triplet creation, entity resolution, and invalidation checks. For a typical medical guideline of 20–30 pages, total cost is about \$0.12.

Processing time averages 45 seconds per document on standard infrastructure. Stage 1 (loading) takes 2 seconds, Stage 2 (chunking) takes 5 seconds for embedding generation, Stage 3 (extraction) takes 25 seconds for dual-track processing, Stage 4 (validation) takes 8 seconds including invalidation checks, and Stage 5 (writing) takes 5 seconds for Neo4j operations.

These costs are acceptable for production medical knowledge systems where accuracy is paramount. The alternative — using cheaper but less accurate extraction methods — leads to inconsistencies that require expensive manual correction and pose patient safety risks.

Best Practices and Lessons Learned

After implementing temporal-aware ingestion in production, several patterns emerged that dramatically impact system quality.

Ingestion Best Practices

Always use **hybrid mode for production systems**. The entity-relationship track provides fast graph traversal while the temporal statement track enables precise time-aware queries. The cost increase from hybrid mode (roughly 1.5x single-track cost) pays for itself in query quality.

Generate embeddings during chunking, not as a separate step. Pre-computing embeddings for chunks enables the system to use them throughout the pipeline: for semantic boundary detection during chunking, for similarity checks during invalidation detection, and for vector search during retrieval. This reuse justifies the upfront embedding cost.

Set temporal precision based on domain needs. Medical research often needs day-level precision because clinical trials have specific start and end dates. Financial news needs hour-level precision for market-moving events. Legal documents often work fine with year-level precision. Don't over-specify precision as it increases storage and query complexity.

Tune invalidation thresholds for your domain. The invalidation agent uses similarity thresholds (0.5 for embeddings, 0.85 for entity resolution) that work well for financial and medical domains. Highly technical domains with specialized terminology may need higher thresholds to avoid false positives.

Ontology Design Patterns

Design ontologies with temporal invalidation in mind. Some entity types naturally invalidate (executive roles, company ownership) while others accumulate (research publications, product launches). Tag entity types with `invalidation_mode`: `REPLACE` (only one valid at a time) or `ACCUMULATE` (multiple can coexist).

Include temporal type hints in relationship definitions. When defining relationship types in your ontology, specify whether they're typically `STATIC` (`ACQUIRED`), `DYNAMIC` (`EMPLOYED_BY`), or `ATEMPORAL` (`FOUNDED_BY`). This guides the temporal agent's extraction.

Version your ontology alongside your graph. When you update entity or relationship types, create new versions rather than modifying existing definitions. This enables temporal queries about what the ontology itself looked like at different points in time.

Performance Optimization

Index temporal properties aggressively. Create composite indexes on (`is_current`, `valid_at`, `expired_at`) to accelerate temporal queries. Point-in-time queries are common and expensive without proper indexing.

Cache invalidation checks. The invalidation agent makes expensive LLM calls. Cache results for common entity pairs to avoid recomputing the same invalidation decisions. In production, this cut invalidation costs by 60%.

Batch write operations. Instead of writing each event individually to Neo4j, batch 50–100 events per transaction. This reduces database connection overhead and improves throughput by 5–10x.

Use async ingestion for real-time sources. Financial news and event streams need low-latency ingestion. Use async document processing with event queues to avoid blocking. Temporal extraction can happen in parallel across chunks.

Monitoring and Observability

Track invalidation rates by entity type. If certain entity types have unusually high invalidation rates (>20% of events), investigate whether extraction is too noisy or whether the domain genuinely changes that rapidly. Adjust extraction prompts or invalidation thresholds accordingly.

Monitor temporal consistency violations. Set up alerts for events where `valid_at > expired_at`, or where `invalid_at < valid_at`. These represent data quality issues that should never occur in production.

Measure extraction quality by temporal type. STATIC events should have precise `valid_at` dates (95%+ accuracy). DYNAMIC events should have reasonable duration ranges. ATEMPORAL facts should rarely be invalidated (<1%). Track these metrics to catch extraction regressions.

Conclusion: Time-Aware Knowledge Graphs for Production Systems

Temporal agents transform knowledge graphs from static snapshots into dynamic, time-aware systems that understand how facts evolve. By implementing dual-track extraction that captures both coarse-grained entity relationships and fine-grained temporal statements, you create a knowledge graph that supports both structural queries and temporal reasoning.

The five-stage ingestion pipeline — document loading with OCR, adaptive semantic chunking, multi-level extraction with temporal agents, resolution and validation

with invalidation detection, and bi-temporal graph writing — ensures that temporal metadata is captured accurately from the start. The invalidation agent automatically detects when new facts supersede old facts, maintaining consistency without manual intervention.

This temporal-aware foundation integrates seamlessly with the GraphOS architecture, enabling retrieval strategies that can answer questions like “What changed?”, “What was true in 2020?”, and “How has this evolved over time?” These capabilities are critical for domains like medical research, financial analysis, and legal discovery where temporal precision matters.

As you build your own temporal knowledge graphs, remember that temporal intelligence starts at ingestion, not retrieval. Capture temporal metadata early, validate it rigorously, and maintain it consistently. The result is a knowledge graph that doesn't just know facts — it knows when those facts were true, why they changed, and how they evolved. That temporal dimension transforms your knowledge graph from a static database into a dynamic system that understands time.

Resources and Further Reading

GraphOS Architecture: [Building Agentic GraphOS: The 16-Layer Architecture Behind Production-Ready Knowledge Graphs](#)

OpenAI Temporal Agents Cookbook: [Temporal Agents with Knowledge Graphs](#)

Entity Resolution with Embeddings: [OpenAI Embeddings Guide](#)

Coming Next: Implementation Guide: Building a Temporal-Aware Ingestion Pipeline

In the next article, we'll move from architecture to hands-on implementation. We'll walk through building a temporal-aware ingestion pipeline step by step, using a **financial news system** as a concrete example. The focus will be on tracking

company acquisitions, executive role changes, and fact invalidation over time, showing how temporal agents work in practice.

Thank you for reading! If you're implementing temporal-aware knowledge graphs in your own systems, I'd love to hear about your experiences. Connect with me on [LinkedIn](#) or follow for more articles on production AI systems.

Conversational AI

Software Development

Technology

Machine Learning

Python



Follow

Written by Akash Goyal

1.1K followers · 233 following

Building Agentic AI applications | Generative AI | LLMs | Agents | NLP | Data Scientist | Machine Learning
<https://www.linkedin.com/in/akash-g-7a5224246/>

Responses (3)



Stellars

What are your thoughts?



James Melvin

Dec 20, 2025



This is a really clean and powerful implementation



1



1 reply

[Reply](#)

Kevin Hughes

Dec 24, 2025



Can you contact me thru contact page of. <http://HereditaryCancer.AI>. Would love to discuss a collaboration.



1 reply

[Reply](#)

Alex Chernikov

Dec 23, 2025



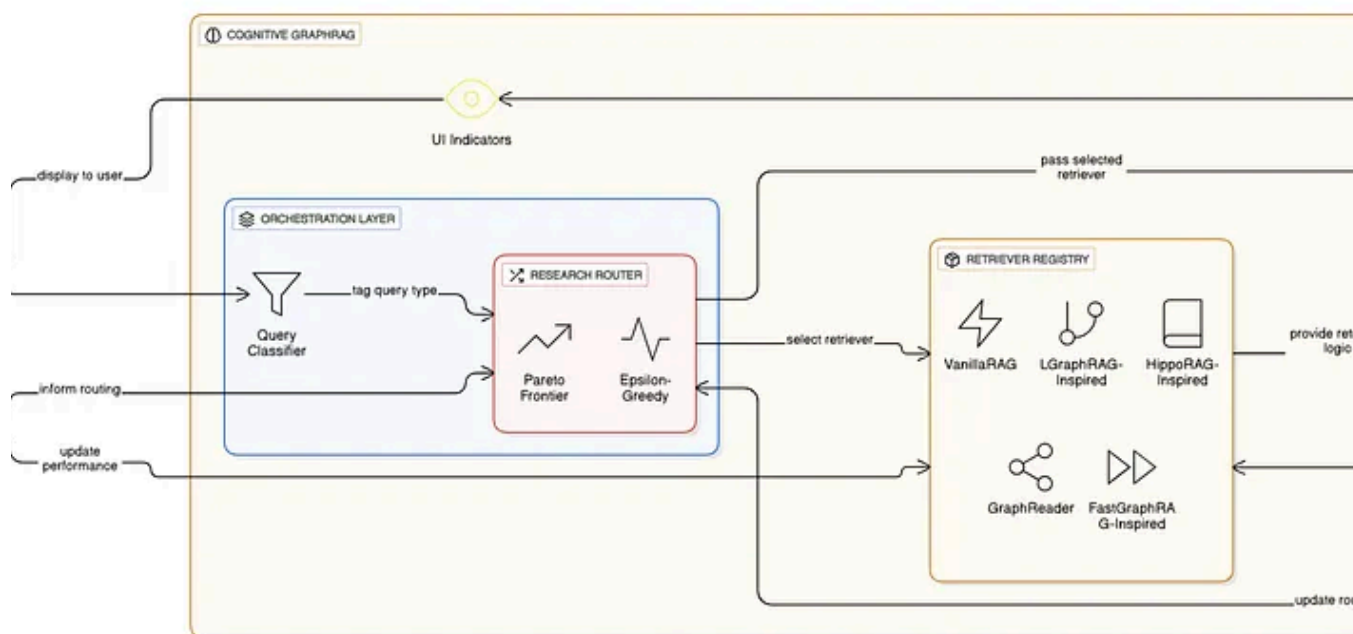
Very impressive article. Regarding the measured processing timings- what infrastructure was used?



2 replies

[Reply](#)

More from Akash Goyal

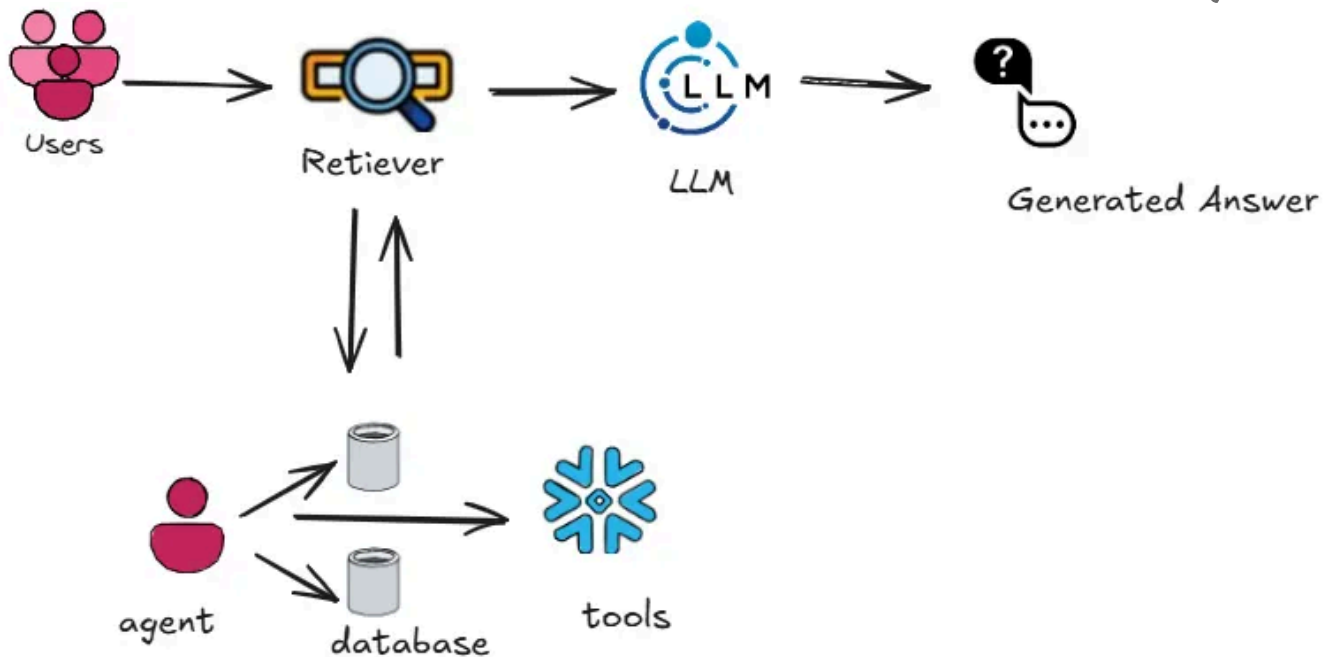


Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval Strategy—Every Time

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform one-size-fits-all GraphRAG.”

★ Dec 13, 2025 🖱 280 💬 3

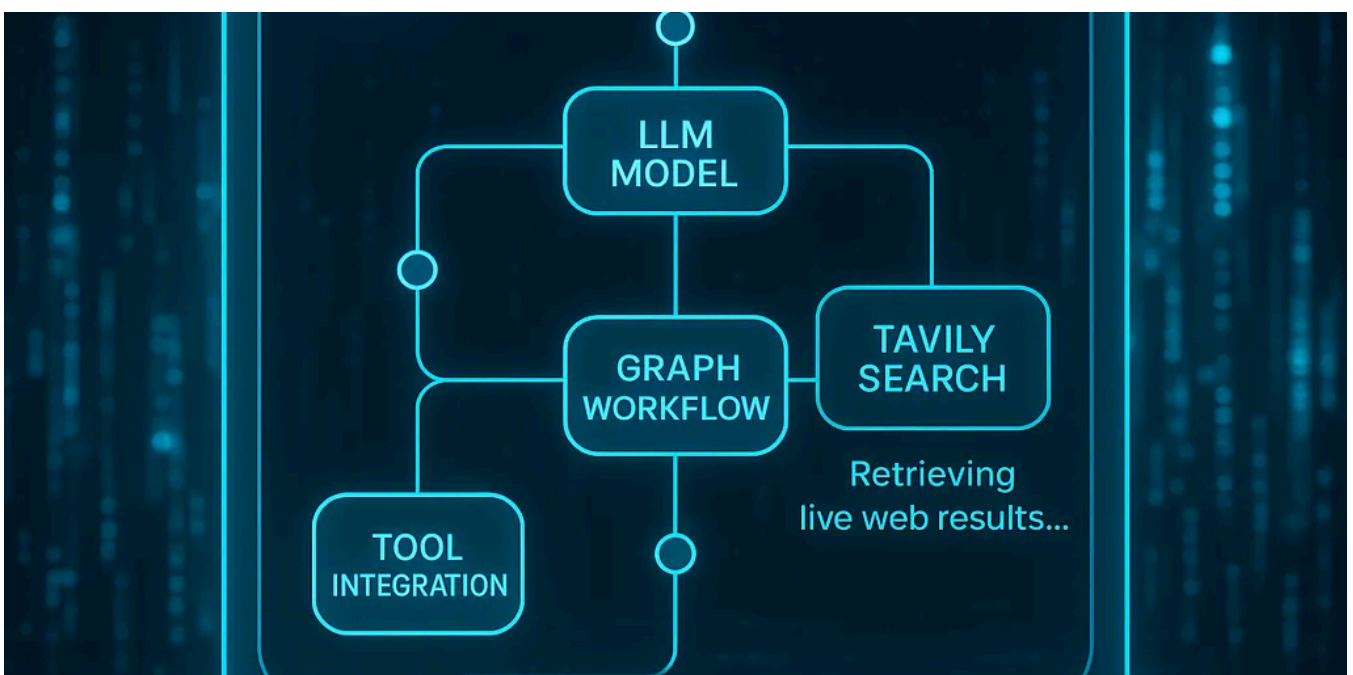


📄 In Generative AI by Akash Goyal

How to make dumb LLMs smarter with agentic retrieval-augmented generation (RAG)

With LangChain, FAISS and Groq

★ Mar 9, 2025 🖱 116 💬 1





In Data Science Collective by Akash Goyal

Beyond Basic Bots: A Guide to Streamlit, LangGraph & Tavily on Hugging Face

In this piece, we'll explore a chatbot application built using Streamlit for the user interface and LangGraph for managing the chatbot's...

★ Apr 1, 2025 🖱 38



Akash Goyal

Building Agentic GraphOS: The 16-Layer Architecture Behind Production-Ready Knowledge Graphs

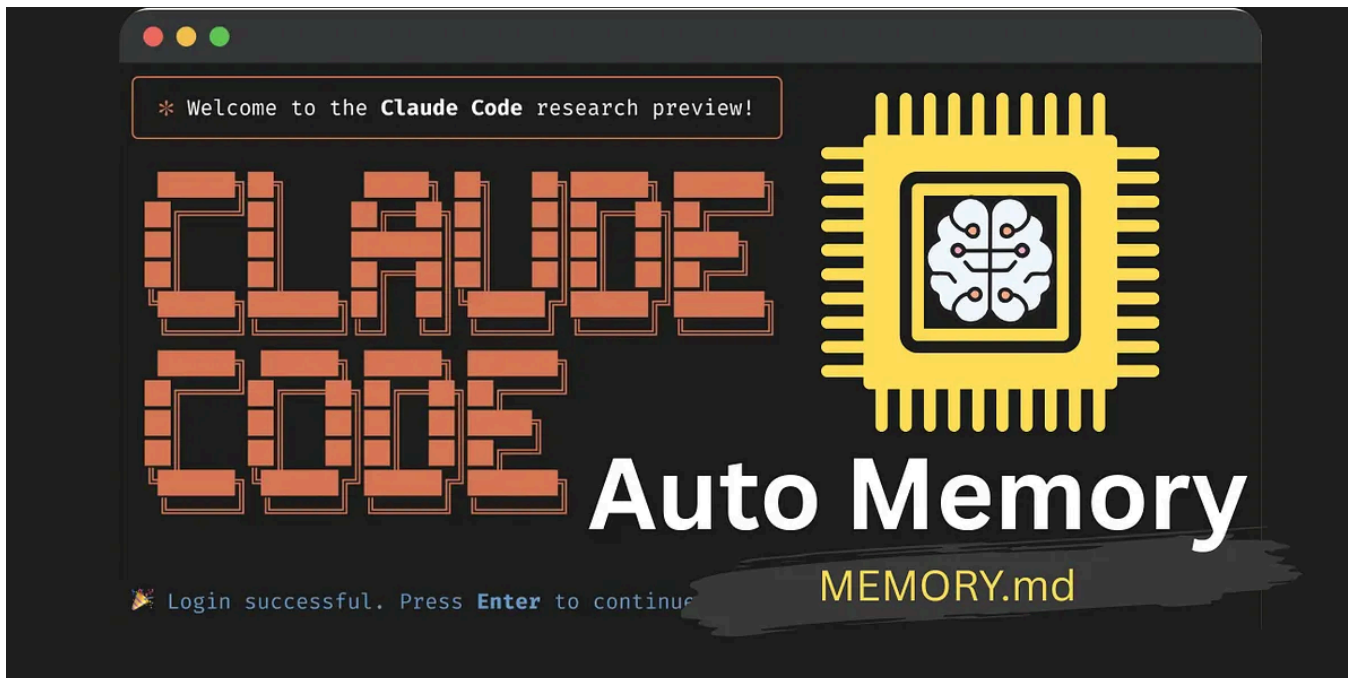
A Cost-Optimized, Fully Observable GraphRAG Operating System Built for Enterprise Production

★ Dec 14, 2025 🖱 648 💬 7



See all from Akash Goyal

Recommended from Medium

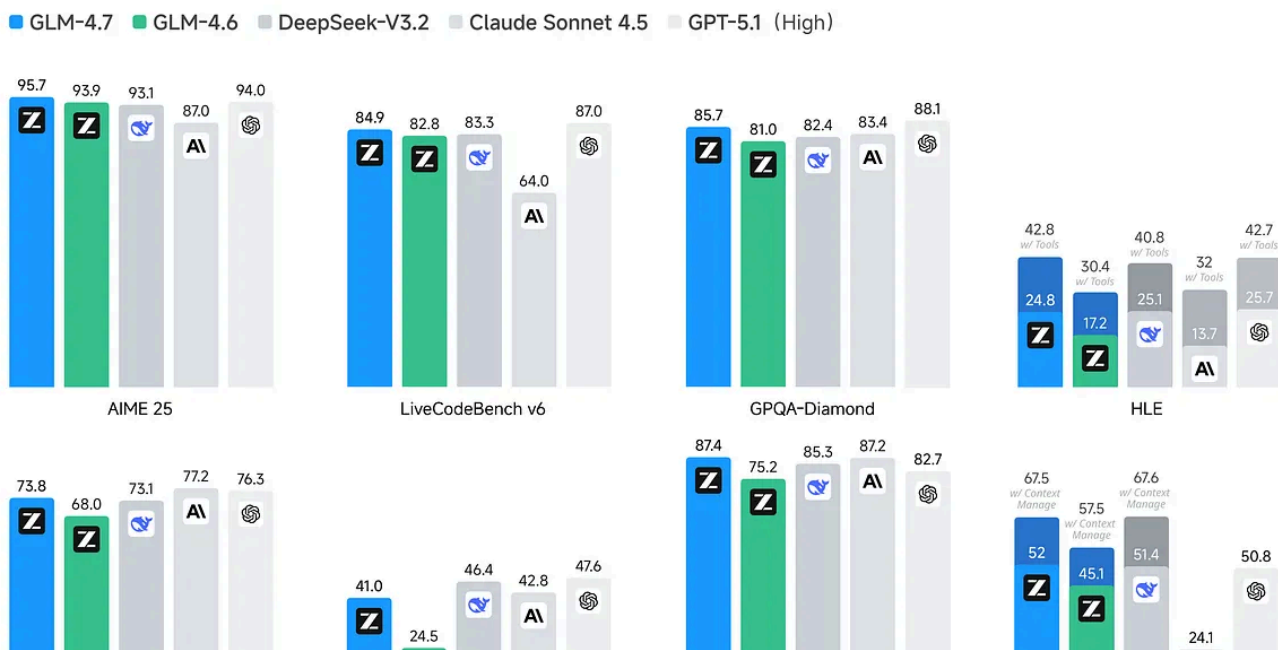


Joe Njenga

Anthropic Just Added Auto-Memory to Claude Code — MEMORY.md (I Tested It)

You don't have to worry about losing your session context anymore. The new auto-memory feature on Claude Code is what you all needed.

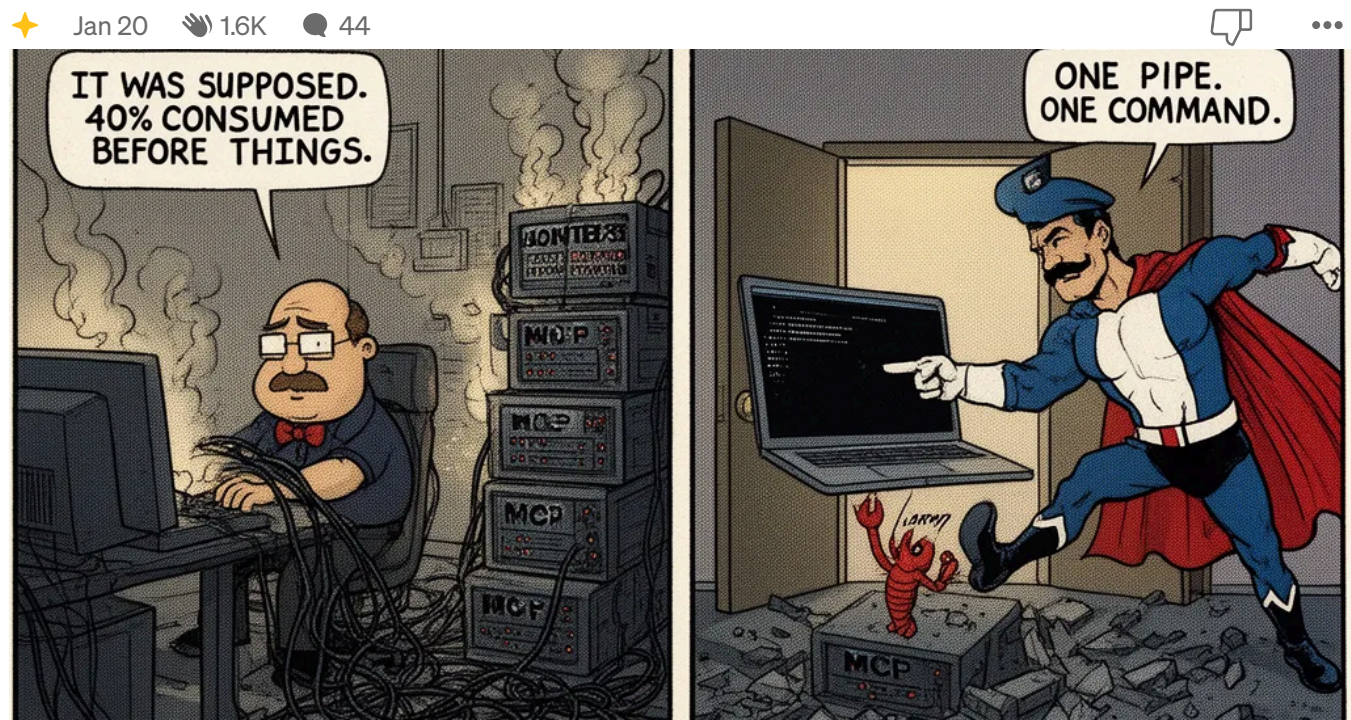
★ Feb 27 🖱️ 463 💬 7



Agent Native

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >\$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you might be wondering...



Phil | Rentier Digital Automation

Why CLIs Beat MCP for AI Agents — And How to Build Your Own CLI Army.

“mcp were a mistake. bash is better.”

Feb 17 1.4K 33

erarchy		
SCOPE		SHARED?
This project, all team members		Yes, via git
Same as above, tidier location		Yes, via git
All your projects, personal		No
This project, just you		No (auto-gitignore)
All users in organization		IT-managed



In Data Science Collective by Paolo Perrone

The Complete Guide to AI Agent Memory Files (CLAUDE.md, AGENTS.md, and Beyond)

One file. Loaded before every conversation. That's all it takes to turn a stateless AI coding assistant into something that actually...

Feb 26



819



11



ADITHYA GIRIDHARAN

Zvec: Alibaba Just Open-Sourced “The SQLite of Vector Databases” — And It’s Blazing Fast

Every decade or so, someone takes a powerful technology that requires dedicated infrastructure and collapses it into a library you can...

Feb 13

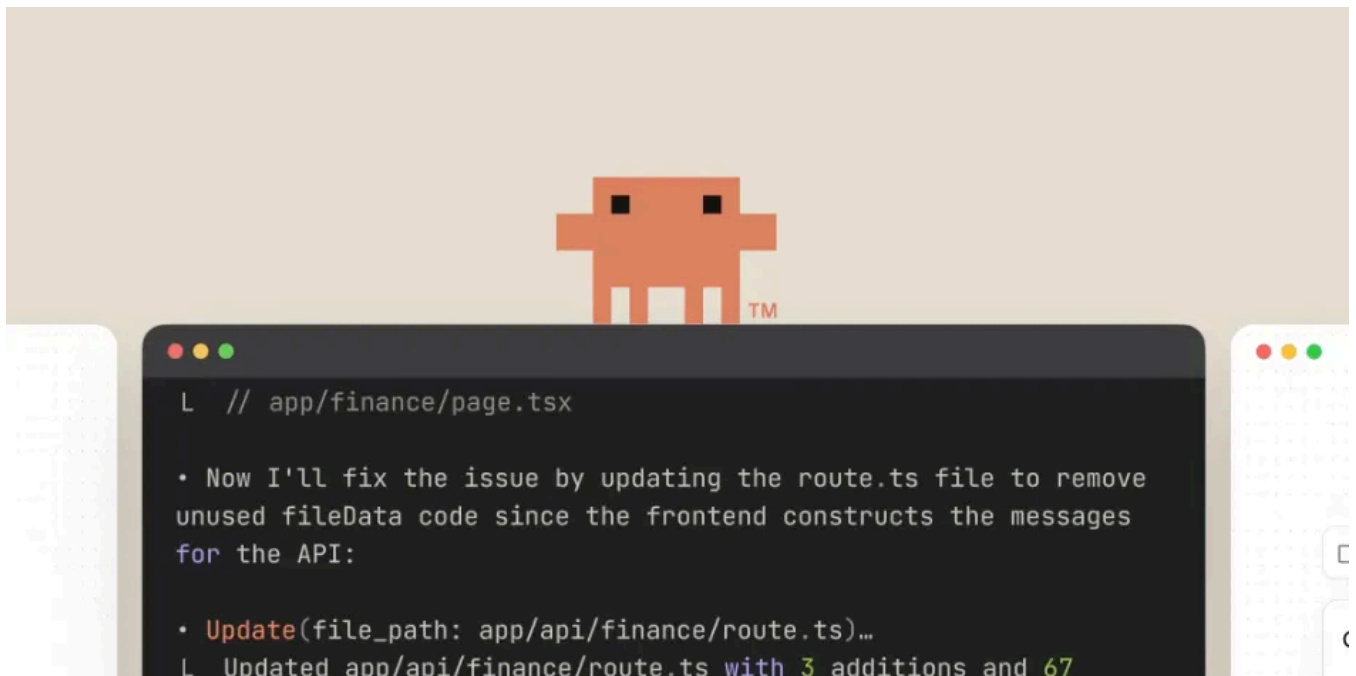


37



3





In AI Advances by Jing Hu

A Lawyer Just Beat 500 Developers at Anthropic's Hackathon

The most valuable skill in AI right now isn't code — and you probably already have it.



Feb 24



1.6K



22



See more recommendations